# WIP: CodeInspector: Automated LLM-Supported CS1-Level Code Assessment

Essa Imhmed, Edgar Ceh-Varela, Ludwig Scherer, George Candal, Ivan Sanjaya
*Department of Mathematical Sciences, Eastern New Mexico University, Portales, NM, USA*
{essa.imhmed, eduardo.ceh, ludwig.scherer, george.candal, ivan.sanjaya}@enmu.edu

***Abstract*—This research-to-practice WIP paper presents *CodeInspector*—an automated assessment system for CS1-level Java assignments. Designed for scalability and modularity, *CodeInspector* streamlines and standardizes code assessment, promoting a more equitable learning environment. Our preliminary findings suggest that *CodeInspector* is promising. It reduces instructors' assessment workload while providing students with immediate, formative feedback that supports an iterative learning process. Additionally, *CodeInspector* offers students insights into their performance through error-weighted densities benchmarked against class-wide averages.**

***Index Terms*—automated assessment, feedback, code quality, software testing and automation, large language models.**

## I. Introduction

Timely and formative feedback is crucial in academia. It allows students to reinforce learning, increase their knowledge, and develop essential skills for more advanced topics. However, manual assessment of student code submissions is time-intensive and delays feedback [1], [2]. Such delays negatively impact student learning and performance, particularly in subsequent assignments.

Instructors often address this issue by solely focusing on code correctness and functionality while neglecting other coding aspects, such as code quality [3], [4]. However, overlooking such crucial coding aspects may encourage students to lean into writing code that functions but with poor code quality, increasing the risk of bugs and maintenance costs. Even worse, instructors may address it by employing multiple teaching assistants in high-enrollment courses, leading to inconsistent grading and feedback [2]. Therefore, these challenges require a more efficient and standardized approach to assessing programming assignments.

In this paper, we introduce ***CodeInspector***—an automated code assessment system for CS1-level Java programming assignments. *CodeInspector* is a scalable and modular solution that leverages open-source tools such as GitHub, Jenkins [5], Apache Ant [6], JUnit [7], CheckStyle [8], PMD [9], and Large Language Models (LLMs) to streamline key aspects of the assessment process, including code quality analysis and unit test generation and execution. It provides students with immediate, formative feedback on correctness, functionality, and code style while reducing instructors' assessment workload.

The contributions of this paper include the following:

1) A scalable and modular automated assessment system leverages open-source tools, allowing customization for various coding aspects and assessment needs.
2) The integration of an LLM for automated test case generation, enhancing the efficiency of assessment workflows.
3) A grading scheme that ensures fair assessment by weighting penalties based on error density and severity rather than the total number of violations.

The remainder of this paper is organized as follows. Section II presents related work relevant to our research. Section III describes the proposed system design. Section IV presents the evaluation results and analysis. Finally, Section V concludes the paper and outlines directions for future work.

## II. Related Work

### A. Code Style Analysis

In academia, the importance of code quality cannot be overstated. Although problem-solving and algorithm implementation often take precedence, the clarity and readability of the code are crucial for effective collaboration and maintenance in software development [10]. Despite this, many programming courses do not adequately emphasize code quality [3]. This is often because the time-consuming nature of manual review forces instructors to focus on assignments that assess problem-solving and algorithmic skills rather than coding standards [10], [11].

The inclusion of code analysis tools in academic settings, as explored in [12]–[14], provides a systematic approach for gaining insights into student code quality and performance. Albluwi and Salter [15] highlighted the effectiveness of feedback from static analysis tools in reducing style errors over time, indicating that these tools can play a vital role in improving code quality education. These academic efforts to highlight and improve code quality through teaching strategies and static analysis tools are pivotal. They enhance students' coding skills and prepare them for software development's collaborative and evolving nature in academia and the professional world.

### B. Large Language Models in Test Case Generation

LLM integration in programming education shows significant promise. LLMs can automate test case generation from software requirements or user stories, replacing the traditional manual approach, which is error-prone and labor-intensive. This automation improves turnaround times and reduces the

need for human oversight [16]. LLMs can also foster an interactive learning environment by delivering real-time feedback, accelerating students' iterative coding process, and reinforcing software testing principles [17], [18]. Their scalability makes them particularly valuable in large programming courses, ensuring consistent and comprehensive test coverage across diverse projects. Furthermore, custom applications of LLMs support various assessment formats, facilitating innovative pedagogical approaches such as adaptive testing [19]. By catching errors early and improving software reliability, LLM-driven test case generation enhances student learning and aligns with best practices in modern software development.

### C. Automated Code Assessment in Education

Several automated code assessment tools and techniques have been proposed for educational purposes. Cipriano et al. [20] of Lusófona University proposed an automatic assessment tool used across several computer science courses. The proposed command-line tool leverages Apache Maven and Unit testing to automate software build and functionality testing. Buffardi & Edwards [21] utilized Web-CAT to automate the evaluation of student code submissions for an introductory programming course at Virginia Tech, allowing students to repeatedly revise and re-submit their code for feedback on their code implementation and functionality. Heckman & King [22] of North Caloraina State University proposed a system that utilizes Jenkins and GitHub for code assessment.

Several studies [23]–[26] utilized unit testing to evaluate student code for correctness and functionality. Insa and Silva [27] provided black-box and white-box testing APIs for code assessment, along with a feature that allows students to compare the output of their code with the expected output. Peveler et al. [26] used several assessment techniques, including the Linux diff tool for output comparison, checks for memory leaks, and performing static code analysis.

## III. System Design

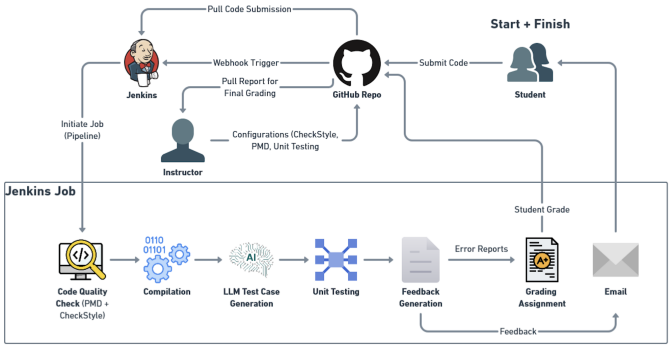


Fig. 1: *CodeInspector* design.

Figure 1 shows the architectural design of the proposed system. *CodeInspector* integrates Jenkins as its central component and LLM for test case generation. The workflow pipeline of the main components shown in the figure is discussed below.

### A. Code Submission

*CodeInspector* offers a web-based front-end interface that allows students to submit their code for evaluation. Students upload their code as a Java file or write it directly using an integrated code editor. The code is then automatically submitted to the instructor's GitHub repository.

### B. Test Case Generation

The system uses LLM to generate unit tests designed to evaluate the correctness of student code. We implemented an API using the Flask[1] framework, enabling communication between *CodeInspector* and the LLM. The open-source tool Ollama[2] executes the LLM locally, eliminating reliance on cloud-based processing. Specifically, we employ Meta's ollama3.2[3]. LLaMA (Large Language Model Meta AI) [28] is a family of state-of-the-art open-weight LLMs that balance computational efficiency with advanced natural language processing capabilities, offering researchers and developers access to powerful AI tools without extensive computational resources. The API accepts a Java file containing an instructor's solution to a programming assignment. The LLM uses the file and a carefully designed prompt specifying rules to generate test cases using the JUnit framework. This process produces a Java file containing test cases for expected inputs and edge cases, which is returned to *CodeInspector* as the API response.

### C. Code Assessment

When students commit their code to the instructor's GitHub repository, GitHub triggers the Jenkins pipeline job for *CodeInspector*, thereby automating the entire assessment process. The pipeline then performs several steps to streamline the build and assessment process. First, it downloads dependencies to set up the execution environment. Next, it retrieves the student code from the GitHub repository, an Apache Ant script for build automation, and the LLM-generated test file. Finally, Jenkins executes the Ant script to compile the code, run unit testing against the executable, and perform the style analysis on the source code.

We selected the JUnit framework for black-box testing and used CheckStyle and PMD for code style analysis. Both CheckStyle and PMD are highly scalable and configurable, enabling instructors to select which checks should be considered when analyzing students' code. To ensure relevance, we configured these tools to enforce a small set of coding standards suitable for novice programmers. Due to space limitations, Table I provides a very brief description of the coding standards from the grading rubric used for the Java code submissions; for a detailed description, see [12], [14].

Additionally, PMD and CheckStyle enable instructors to customize style error severity levels. In PMD, severity is defined on a scale from 1 to 4, where one is the most severe, while in CheckStyle, severity levels are categorized as

[1]https://flask.palletsprojects.com/en/stable/
[2]https://ollama.com
[3]https://www.llama.com/llama-downloads/

TABLE I: Coding standards grading criterion, from [12].

| *Criterion* | *Description* |
|---|---|
| Javadoc Documentation | The code is well documented, including proper JavaDoc tags that comment on the author's name, date, program's task, and JavaDoc methods that effectively document the program's methods. |
| Comments | Comments specifying the objective of each method and control flow constructs are present and meaningful. |
| Naming Conventions | The program names, methods, and variables are descriptive and adhere to coding standards. |
| Whitespace | There is proper indentation and white space; code lines are not excessively long (less than 100 characters). |

error, warning, or info. Once the unit test execution and code style analysis are complete, assessment reports are generated detailing the number of passed and failed tests and identified style errors categorized by severity.

### D. Feedback Generation

At this stage, *CodeInspector* consumes the detailed reports generated from unit test execution and style analysis to create more formative feedback, which is then emailed to the corresponding student. This report lists aggregated counts of style errors by severity and details the first three detected errors, while hiding details of other violations until earlier errors are addressed. It also includes test case outcomes and an error-weighted density metric, providing students with insights into their performance relative to their peers.

By providing streamlined feedback via email, *CodeInspector* allows students to revise their code and resubmit it for further assessment. This structured approach minimizes cognitive overload, encourages incremental progress, and promotes iterative refinement rather than dependence. Moreover, it enables instructors to track student progress more efficiently. Future improvements may include submission tokens to limit excessive resubmissions and reduce over-reliance on *CodeInspector* for debugging.

### E. Automated Grading

In the final stage of the Jenkins pipeline, *CodeInspector* grades student submissions using predefined, tiered criteria that range from Excellent to Unsatisfactory. A percentage threshold determines each tier and contributes a specific portion to the total grade for the corresponding criterion (see [12]). The grading criteria are outlined below.

*1) Requirements:* This criterion evaluates how thoroughly the submitted code meets the assignment's functional requirements. The grade calculation for this criterion is straightforward. *CodeInspector* runs JUnit tests and then calculates the pass rate to determine which points should be assigned from the tiered rubric.

*2) Coding Standards:* Unlike the requirements criterion, style error distributions can vary significantly between submissions. To ensure fairness when grading this criterion, we adopt a penalty-based system rather than threshold scoring

**Algorithm 1:** Error-weighted density grading.

**Input:** Max_Grade, Min_Grade, Max_Deduction, Penalty_per_Category, Errors_per_Category, LOC
**Output:** Final_Grade

```
Let Total_Penalty ← 0;
foreach category ∈ Errors_per_Category[category] do
    Absolute_Penalty ← Penalty_per_Category[category] × Errors_per_Category[category];
    Error_Density ← Errors_per_Category[category] ÷ LOC;
    Adjusted_Penalty ← Absolute_Penalty × Error_Density;
    Total_Penalty ← Total_Penalty + Adjusted_Penalty;
end
Final_Grade ← max(Max_Grade - min(Total_Penalty, Max_Deduction), Min_Grade);
return Final_Grade;
```

or simply counting total errors, thereby penalizing each style error according to its severity. Additionally, code size is considered since smaller or incomplete submissions may have fewer violations than larger ones. Penalizing solely based on total violations without considering code size is unfair and may discourage novice programmers from writing more extensive code to avoid higher penalties. Therefore, our grading method bases penalties on error-weighted density rather than total violations.

Algorithm 1 implements the grading scheme used by *CodeInspector* to determine the code quality grade of a submission. The process begins by computing the absolute penalty for each error category, calculated as the total style violations multiplied by the corresponding penalty weight. Next, the algorithm calculates the error density to normalize the penalty relative to code size; error density is defined as the number of errors per line of code (LOC). Each category's adjusted penalty is then obtained by scaling the absolute penalty by the error density, ensuring that students with larger codebases are not unfairly penalized for having more violations. The total penalty is calculated by summing the adjusted penalties and is then subtracted from the maximum grade, with the deduction capped at a maximum limit to prevent excessive penalties. Finally, the resulting grade is compared to a predefined minimum threshold, and the higher value is assigned as the student's final grade, ensuring that grades do not fall below this minimum.

*3) Runtime:* Like the requirements criterion, *CodeInspector* runs unit and edge case tests to assess the code's correctness and behavior. The grade assigned from the grading rubric corresponds directly to the test pass rate.

## IV. Results and Discussion

In the ENMU Computer Science program, CSCI 1210: the Computer Programming Fundamentals course is taught in Java and delivered in a hybrid format, combining in-person instruction with asynchronous distance learning. To evaluate *CodeInspector*, students were assigned a benchmark task that required them to implement a method that converts the rank and suit of a playing card from shorthand notation to its complete descriptive form. A total of 19 submissions, comprising 1,315 lines of code, were submitted to *CodeInspector* to assess code quality and correctness, with a copy of each assessment result automatically emailed to the corresponding student.

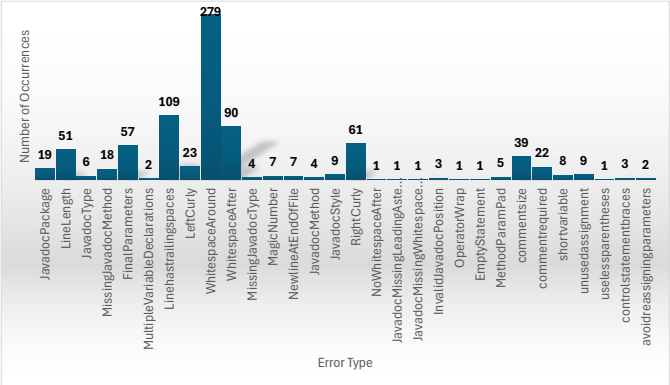


Fig. 2: Frequency distribution of detected style errors.

```
========================   CODING STANDARDS
=================================================

CHECKSTYLE ---------------------------------------------

An overall of 121 violations were detected by CheckStyle.

Breakdown by severity level:

 Severity: info - 15 violations
  Most frequent category: regexp (11 occurrences)
  Most frequent type: RegexpSingleline (11 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: Line has trailing spaces.
 Severity: warning - 98 violations
  Most frequent category: whitespace (98 occurrences)
  Most frequent type: WhitespaceAroundCheck (77 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: 'if' is not followed by whitespace.
 Severity: error - 8 violations
  Most frequent category: checks (3 occurrences)
  Most frequent type: FinalParametersCheck (3 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: Parameter rank should be final.

Overall weighted CheckStyle error density: 0.02758
This places you in the 83rd percentile of submissions in the database.
That is, 17% of submissions had a higher CheckStyle error density than yours.
```

Fig. 3: Style errors detected by CheckStyle.

Figure 2 shows the frequency of coding standards violations identified by CheckStyle and PMD within *CodeInspector*, totaling 843 style errors. Of these, *"WhiteSpaceAround," "LineHasTrailingSpaces,"* and *"WhiteSpaceAfter"* were the most frequently occurring violations in all submissions. While whitespace errors do not affect code functionality, they reduce code readability and hinder collaboration, making it harder for reviewers to understand changes during code reviews and maintenance.

Figure 3 presents a portion of a report generated by *CodeInspector* for a student's code submission, showing style errors detected by CheckStyle. Due to space limitations, violations identified by PMD are omitted. Of the 121 errors reported by CheckStyle, 15 were classified as "info," 98 as "warning," and eight as "error," highlighting issues that range from minor suggestions to critical problems requiring immediate attention. Notably, the most frequent violations across all categories were *"WhiteSpaceAround"* and *"LineHasTrailingSpaces"* (88 occurrences), as well as *"FinalParametersCheck"* (3 occurrences), which highlights a recurring pattern of non-final parameters in methods, a common pitfall in early-stage programming education. This report demonstrates the system's ability to provide granular and actionable feedback, enabling students to identify areas for stylistic improvement and iteratively refine their code based on clear, rule-based guidelines. Additionally, the report's structured breakdown by severity and category enables instructors to prioritize feedback.

*CodeInspector*'s integration of CheckStyle further demonstrates its utility in contextualizing student performance. For example, the reported error-weighted density (0.02758) places this submission in the 83rd percentile of analyzed submissions, indicating a higher rate of code style violations than 83% of the sumissions, reflecting weaker adherence to coding standards. This comparative metric underscores the system's ability to deliver formative feedback and benchmarking insights, fostering self-awareness and motivating continuous improvement. Such automated, data-driven assessments also reduce instructors' grading burdens while ensuring consistent and objective evaluation, which is a critical step toward scalable, high-quality coding education.

```
============================   REQUIREMENTS + RUNTIME
=====================================

JUNIT ---------------------------------------------

An overall of 5 out of 11 unit tests failed.

The following are the ones that failed:
    Test: testGetCardDescription_FiveOfDiamonds failed with the error:
expected:<[Five] of Diamonds> but was:<[5] of Diamonds>
    Test: testGetCardDescription_InvalidRank failed with the error:
expected:<[Invalid card rank]> but was:<[ of Diamonds]>
    Test: testGetCardDescription_InvalidSuit failed with the error:
expected:<[Invalid card suit]> but was:<[Ace of ]>
    Test: testGetCardDescription_EmptyRank failed with the error:
expected:<[Invalid card rank]> but was:<[ of Diamonds]>
    Test: testGetCardDescription_EmptySuit failed with the error:
expected:<[Invalid card suit]> but was:<[Ace of ]>

================================  FINAL SCORE
=========================================
    - Requirements: 36
    - Coding Standards: 9
    - Runtime: 12
=================================================================================
===================
    - Overall: 57
========================================= END REPORT
```

Fig. 4: Unit testing results and final scores from the generated feedback.

Figure 4 presents another portion of the same feedback report shown in Figure 3, highlighting formative feedback generated by *CodeInspector* to help students address functional issues in their code. The report demonstrates the effectiveness of *CodeInspector*'s automated testing framework powered by JUnit and LLM-generated test cases in identifying correctness issues and assigning holistic scores to student submissions. For this particular submission, 11 unit test cases were executed, of which six passed successfully, while five failed due to various issues detailed in the feedback. One such issue involved a discrepancy between the expected output string ("[Five] of Diamonds") and the actual output ("[5] of Diamonds"). This granular feedback exemplifies how *CodeInspector* leverages LLM-driven test generation to pinpoint logical errors, enabling students to refine their code against precise, real-world expectations iteratively.

Moreover, the final score breakdown further underscores *CodeInspector*'s modular evaluation design, which balances multiple assessment criteria (e.g., requirements fulfillment, coding standards, runtime performance) to reflect holistic competency. For this particular submission, *CodeInspector*

assigned an overall score of 57, demonstrating its ability to synthesize diverse evaluative dimensions into actionable insights. This structured, automated scoring mechanism streamlines grading while ensuring transparency and consistency, aligning with pedagogical goals of fostering technical rigor and supporting iterative improvement in student work.

## V. Conclusion and Future Work

*CodeInspector* represents a transformative advancement in automated code assessment, seamlessly integrating open-source tools to deliver scalable, modular, and pedagogically aligned evaluations. By leveraging GitHub-triggered Jenkins pipelines and Apache Ant scripts, the system automates the entire build and assessment lifecycle, ensuring consistency and efficiency at scale. The strategic selection of configurable tools like CheckStyle and PMD for code style analysis, JUnit for automated unit testing, and LLMs for test case generation, reinforces *CodeInspector*'s commitment to fostering foundational coding practices while minimizing cognitive overload.

Our preliminary findings demonstrate that *CodeInspector* reduces instructors' grading burdens and empowers students through granular, actionable feedback. The holistic scoring framework, combining requirements fulfillment, coding standards, and runtime performance, reflects a nuanced understanding of competency, while efficiency metrics remain customizable to suit diverse curricula. *CodeInspector* paves the way for adaptive, data-driven education. Its modular design enables seamless updates to accommodate evolving standards and emerging technologies, ensuring long-term relevance.

Future work will focus on fully automating test case generation within *CodeInspector*, eliminating the need for manual prompt design or other forms of human intervention. It will also explore deeper integration of AI-driven feedback personalization and improved efficiency. Moreover, we will evaluate *CodeInspector* across multiple student groups to collect user feedback and gain deeper insights into its educational effectiveness.

## Acknowledgment

This project is funded by Eastern New Mexico University through the Faculty Research and Instructional Development Grant program.

## References

[1] Andre LC Barczak, Anuradha Mathrani, Binglan Han, and Napoleon H Reyes. Automated assessment system for programming courses: a case study for teaching data structures and algorithms. *Educational technology research and development*, 71(6):2365–2388, 2023.

[2] Ramon Lawrence, Sarah Foss, and Tatiana Urazova. Evaluation of submission limits and regression penalties to improve student behavior with automatic assessment systems. *ACM Trans. Comput. Educ.*, 23(3), June 2023.

[3] Hsi-Min Chen, Wei-Han Chen, and Chi-Chen Lee. An automated assessment system for analysis of coding convention violations in java programming assignments. *J. Inf. Sci. Eng.*, 34:1203–1221, 2018.

[4] Markus Hofbauer, Christoph Bachhuber, Christopher Kuhn, and Eckehard Steinbach. Teaching software engineering as programming over time. In *2022 IEEE/ACM 4th International Workshop on Software Engineering Education for the Next Generation (SEENG)*, pages 51–58, 2022.

[5] Jenkins. https://www.jenkins.io, 2025. accessed: 2025-07-26.

[6] Apache ant. https://ant.apache.org, 2025. accessed: 2025-07-26.

[7] Junit. https://junit.org/junit4/, 2025. accessed: 2025-07-26.

[8] Checkstyle. https://checkstyle.sourceforge.io, 2025. accessed: 2025-07-26.

[9] Pmd. https://pmd.github.io, 2025. accessed: 2025-07-26.

[10] Oscar Karnalim and Simon. Promoting code quality via automated feedback on student submissions. In *2021 IEEE Frontiers in Education Conference (FIE)*, pages 1–5, 2021.

[11] Oscar Karnalim, Simon, and William Chivers. Work-in-progress: Code quality issues of computing undergraduates. In *2022 IEEE Global Engineering Education Conference (EDUCON)*, pages 1734–1736, 2022.

[12] Essa Imhmed, Edgar Ceh-Varela, and Scott Kilgore. Identifying code quality issues for undergraduate students using static analysis and nlp. In *2023 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 1527–1533, 2023.

[13] Edgar Ceh-Varela and Essa Imhmed. Investigating freshmen students' coding standards challenges using nlp techniques. In Gerhard-Wilhelm Weber, Jose Francisco Martinez Trinidad, Michael Sheng, Raghavendra Ramachand, Latika Kharb, and Deepak Chahal, editors, *Information, Communication and Computing Technology*, pages 3–15, Cham, 2025. Springer Nature Switzerland.

[14] Essa Imhmed, Edgar Ceh-Varela, Hashim Abu-Gellban, and Scott Kilgore. Fostering code quality practices among undergraduate novice programmers. *J. Comput. Sci. Coll.*, 39(7):21–32, April 2024.

[15] Ibrahim Albluwi and Joseph Salter. Using static analysis tools for analyzing student behavior in an introductory programming course. *Jordanian Journal of Computers and Information Technology (JJCIT)*, 6(3):215–233, 2020.

[16] Nischal Ashok Kumar and Andrew Lan. Using large language models for student-code guided test case generation in computer science education. *arXiv preprint arXiv:2402.07081*, 2024.

[17] Weimin Xiong, Yiwen Guo, and Hao Chen. The program testing ability of large language models for code. *arXiv preprint arXiv:2310.05727*, 2023.

[18] Felix Dobslaw, Robert Feldt, Juyeon Yoon, and Shin Yoo. Challenges in testing large language model based software: A faceted taxonomy. *arXiv preprint arXiv:2503.00481*, 2025.

[19] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1643–1655, 2024.

[20] Bruno Pereira Cipriano, Nuno Fachada, and Pedro Alves. Drop project: An automatic assessment tool for programming assignments. *SoftwareX*, 18:101079, 2022.

[21] Kevin Buffardi and Stephen H. Edwards. Effective and ineffective software testing behaviors by novice programmers. ICER '13, page 83–90, New York, NY, USA, 2013. Association for Computing Machinery.

[22] Sarah Heckman and Jason King. Developing software engineering skills using real tools for automated grading. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, SIGCSE '18, page 794–799, New York, NY, USA, 2018. Association for Computing Machinery.

[23] Mario Amelung, Katrin Krieger, and Dietmar Rösner. E-assessment as a service. *IEEE Transactions on Learning Technologies*, 4(2):162–174, 2011.

[24] Sebastian Pape, Julian Flake, Andreas Beckmann, and Jan Jürjens. Stage - software tool for automatic grading of testing exercises - case study paper. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 491–500, 2016.

[25] Brad Vander Zanden and Michael W Berry. Improving automatic code assessment. *Journal of Computing Sciences in Colleges*, 29(2):162–168, 2013.

[26] Matthew Peveler, Jeramey Tyler, Samuel Breese, Barbara Cutler, and Ana Milanova. Submitty: An open source, highly-configurable platform for grading of programming assignments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, pages 641–641, 2017.

[27] David Insa and Josep Silva. Automatic assessment of java code. *Computer Languages, Systems & Structures*, 53:59–72, 2018.

[28] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.